# Machine Learning e Visão Computacional [T1]

Situação de Aprendizagem (Projeto Avaliativo) - Módulo 2 - Semana 14

## SUMÁRIO


1 CONTEXTUALIZAÇÃO 1
2 DESAFIO 1
3 ENTREGA 2
4 REQUISITOS DA APLICAÇÃO 2
5 ROTEIRO DA APLICAÇÃO 2
    5.1 FORMATO DO SISTEMA 2
    5.2 DOCUMENTAÇÃO NO README.MD 3
    5.3 USO DO TRELLO 3
    5.4 USO DO GITHUB 3
    5.5 GRAVAÇÃO DE VÍDEO 4
6 CRITÉRIOS DE AVALIAÇÃO 4
7 PLANO DE PROJETO 5


## 1 CONTEXTUALIZAÇÃO

A evolução da Visão Computacional moderna não se limita apenas a saber "o que" está em uma imagem (Classificação), mas "onde" e "quantos" objetos estão presentes em tempo real (Detecção de Objetos). Ferramentas estado-da-arte, como a família de modelos YOLO (You Only Look Once), revolucionaram indústrias que vão desde veículos autônomos até a segurança no trabalho, permitindo inferências ultra rápidas.

No mercado de trabalho atual, no entanto, treinar um modelo em um Jupyter Notebook não é mais suficiente. O Engenheiro de IA moderno precisa saber empacotar esse modelo em uma API RESTful para que outros sistemas da empresa possam consumi-lo, além de utilizar metodologias ágeis (Kanban/Scrum) e versionamento (Git) para gerenciar o ciclo de vida desse software.

## 2 DESAFIO

Você foi escalado como Engenheiro de Machine Learning em uma startup de Inteligência Artificial. Seu desafio técnico é construir um sistema backend completo de detecção de objetos, desde a pesquisa dos dados até o deploy da API.

Para isso, **a escolha do dataset será livre!** Vocês poderão buscar conjuntos de dados (no Kaggle, Roboflow, etc.) que resolvam um problema do interesse de vocês (ex: peças de carro,

itens de supermercado, animais silvestres). A única regra de negócio é: **o dataset escolhido deve possuir pelo menos 10 classes diferentes de objetos** com uma boa variedade de imagens para testar a robustez do modelo em cenários diversos.

Sua tarefa consiste em treinar (ou aplicar *fine-tuning*) em um modelo da família YOLO (YOLOv8, YOLOv9, YOLO11, etc.) para que ele aprenda a detectar perfeitamente as 10 (ou mais) classes do seu dataset. Após o modelo estar treinado e validado, você deverá integrá-lo a uma API construída com FastAPI. Esta API deverá receber uma imagem, processá-la e retornar um JSON estruturado com as detecções.

***Atenção:*** Se o trabalho for realizado em **Squads (Equipes)**, o desafio é ampliado para um *Benchmark*. Cada integrante do grupo deverá fazer uma versão diferente do YOLO (ex: Aluno A com v8, Aluno B com v9, Aluno C com v11) sobre as mesmas 10 classes. A equipe deverá comparar os resultados de acertos e velocidade, e justificar qual modelo foi escolhido como o "campeão" para ser embutido na API final do grupo. Todos devem realizar a entrega.

## 3 RESULTADOS ESPERADOS (ENTREGA)

O código deverá ser inserido e versionado no GitHub, o vídeo deverá ser inserido no Google Drive/Youtube em modo leitor para qualquer pessoa com o link, e o planejamento e organização deverá ser realizado no Trello(ou outro Kanban, compartilhar com email: welington.souza@edu.sc.senai.br). Os três links deverão ser disponibilizados na tarefa Módulo 2 - Projeto Avaliativo, presente na semana 14 do AVA até o dia 02/11/2026 às 15h.

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter os links no AVA. Os links não submetidos terão penalidade na nota.

## 4 REQUISITOS DA APLICAÇÃO

- **Aquisição de Dados:** Escolha e preparação de um dataset contendo imagens variadas e, **obrigatoriamente, um mínimo de 10 classes de objetos distintos**.
- **Modelo de IA:** Aplicação em pelo menos uma arquitetura YOLO (v8, v9, v11, etc.)
- **Backend (API):** Desenvolvimento de uma API utilizando o framework FastAPI em Python.

- **Endpoint de Inferência:** A API deve possuir um endpoint (ex: `/detect`) configurado com método `POST` que aceite o upload de um arquivo de imagem (`UploadFile`).
- **Retorno de Dados:** A API *não* deve retornar a imagem desenhada, mas sim um objeto JSON contendo uma lista de detecções estruturadas (classe, confiança e bounding box).
- **Metodologia Ágil:** Uso de um board Kanban (Trello ou outro) para mapear todas as etapas de desenvolvimento da IA e da API.

## 5 ROTEIRO DA APLICAÇÃO

Abaixo segue um detalhamento de como a aplicação deve se comportar e ser documentada.

### 5.1 FORMATO DO SISTEMA

O sistema operará como um microserviço *backend-only*. O usuário (ou sistema cliente) enviará uma imagem contendo os elementos do dataset escolhido via Postman (ou Swagger/docs nativo do FastAPI) e receberá a resposta estruturada.

*Exemplo de formato JSON esperado no retorno:*

```
{
  "status": "success",
  "detections": [
    {
      "class_name": "nome_da_sua_classe_1",
      "confidence": 0.92,
      "box": {"x_min": 150, "y_min": 200, "x_max": 250, "y_max": 300}
    },
    {
      "class_name": "nome_da_sua_classe_2",
      "confidence": 0.85,
      "box": {"x_min": 40, "y_min": 50, "x_max": 100, "y_max": 120}
    }
  ]
}
```

### 5.2 DOCUMENTAÇÃO NO README.MD

Crie um arquivo readme.md no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas.

Algumas dicas interessantes para a documentação:

- Criar um nome para o seu software de detecção;
- Descrever qual dataset foi escolhido, quais são as 10 (ou mais) classes e o desafio que ele resolve;
- Explicar qual versão do YOLO foi utilizada e por que (se em squad, colocar a tabela de comparação/benchmark entre os modelos testados);
- Descrever como instalar as dependências (`requirements.txt`) e executar a API localmente (`uvicorn main:app --reload`);
- Exemplo de *payload* (JSON) de saída.

### 5.3 USO DO TRELLO

Crie um quadro no seu Trello(ou outro) pessoal, utilizando a metodologia kanban (backlog, todo, doing, blocked, review e done). Organize as tarefas a serem realizadas em cartões (cards) e os arraste a cada etapa (ex: Escolher e Limpar Dataset de 10 classes, Realizar Fine-Tuning do YOLO, Criar Endpoint, Testar Postman). Note que você pode fazer um paralelo com a metodologia do GitFlow, criando cartões com o mesmo nome, por exemplo `feature/create-fastapi-endpoint`. Não esqueça de habilitar o quadro para modo público e colocar o link no readme.md e na submissão no AVA.

### 5.4 USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir commits com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por commit.
- Crie uma branch “develop” para concentrar os merges enquanto estiver desenvolvendo as funcionalidades.
- Utilizar feature branches a partir da develop para cada tarefa (ex: `feature/dataset-prep`, `feature/yolo-finetuning`, `feature/api-setup`).
- O código final do projeto deve estar todo “mergeado” no branch “main”.

### 5.5 GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo máximo de 7 minutos, abordando os seguintes questionamentos:

- Qual dataset vocês escolheram (mostre as 10 classes) e qual o objetivo do sistema?
- Faça a demonstração de funcionamento (envio de uma imagem no Swagger/Postman e recebimento do JSON).
- Como foi o processo do YOLO (e o benchmark das versões, caso tenha feito em equipe)? O modelo generalizou bem para imagens variadas?
- Mostrar os testes do dataset.
- Como você organizou as tarefas no Trello antes de começar a desenvolver?
- Quais as branches do GitHub você criou e quais os objetivos para cada uma?

Você poderá gravar na vertical ou na horizontal. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive ou Youtube em modo leitor para qualquer pessoa com o link.

## 6 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima. Projetos que apresentarem plágio receberão nota 0 (zero).

| Apresentação do Projeto | | | |
|---|---|---|---|
| Nº | Critério de Avaliação | 0 | 2,00 |
| 1 | Realizou a gravação de um vídeo? | Não foi realizada a gravação do vídeo. | Gravou o vídeo e abordou todos os tópicos (dados, API, Kanban, Git e IA). |

| Uso adequado do Trello | | | | |
|---|---|---|---|---|
| Nº | Critério de Avaliação | 0 | 0,25 | 0,50 |
| 2 | Aluno atribuiu adequadamente seu nome no card, bem como especificações da tarefa? | O aluno não criou/atribuiu o card. | Aluno criou o card, mas não colocou seu nome ou comentários especificados nos requisitos. | Aluno criou o card, atribuiu seu nome nele e colocou as especificações de implementação no mesmo. |
| 3 | O aluno movimentou adequadamente o Card no board? | Aluno não movimentou o card no board. | Aluno movimentou o card de maneira inadequada no board. | O aluno movimentou adequadamente o card, respeitando as regras no Kanbanize. |

| Uso adequado do GitHub e Readme.md | | | | |
|---|---|---|---|---|
| Nº | Critério de Avaliação | 0 | 0,25 | 0,50 |
| 5 | Foi aberto um branch separado para cada etapa, com commits claros? | Commits direto na main, sem mensagens claras. | Branchs criadas, mas com commits misturados ou mensagens genéricas. | Branchs separadas (feature/) e commits concisos descrevendo as entregas. |
| 6 | Criou uma documentação com readme.md? | Não criou a documentação. | Documentação muito superficial. | Documentação completa (descrição do dataset, da IA, instrução de execução e payload). |

| Desenvolvimento adequado da Aplicação | | | | |
|---|---|---|---|---|
| Nº | Critério de Avaliação | 0 | 1,00 | 3,00 |

| | | | | |
|---|---|---|---|---|
| 7 | Aquisição de Dados e Treinamento (Fine-Tuning) | Não implementou o YOLO ou usou um dataset com menos de 10 classes. | O modelo não reconhece as 10 classes com clareza ou não justificou a escolha da arquitetura. | Realizou o (fine-tuning) YOLO com sucesso em um dataset com pelo menos 10 classes e testou com imagens variadas (apresentando o benchmark se em squad). |
| 8 | Construção da API (FastAPI) | Não construiu a API. | API construída, mas não recebe a imagem corretamente via POST (`UploadFile`). | Endpoint configurado corretamente para recebimento de arquivos binários em POST e retorno do json com os resultados. |

## 7 PLANO DE PROJETO

Ao construir a aplicação proposta, o aluno estará colocando em prática os aprendizados em:

- Atuar no processo de engenharia de software ágil, utilizando Kanban e GitFlow.
- Selecionar, higienizar e preparar datasets complexos (multiclasse).
- Aplicar rotinas de *fine-tuning* e avaliar o desempenho de algoritmos de detecção de objetos (YOLO).
- Desenvolver interfaces de integração (APIs) modernas com FastAPI para servir Modelos de IA (MLOps Básico), abstraindo o processamento matricial para um retorno leve e universal via JSON.